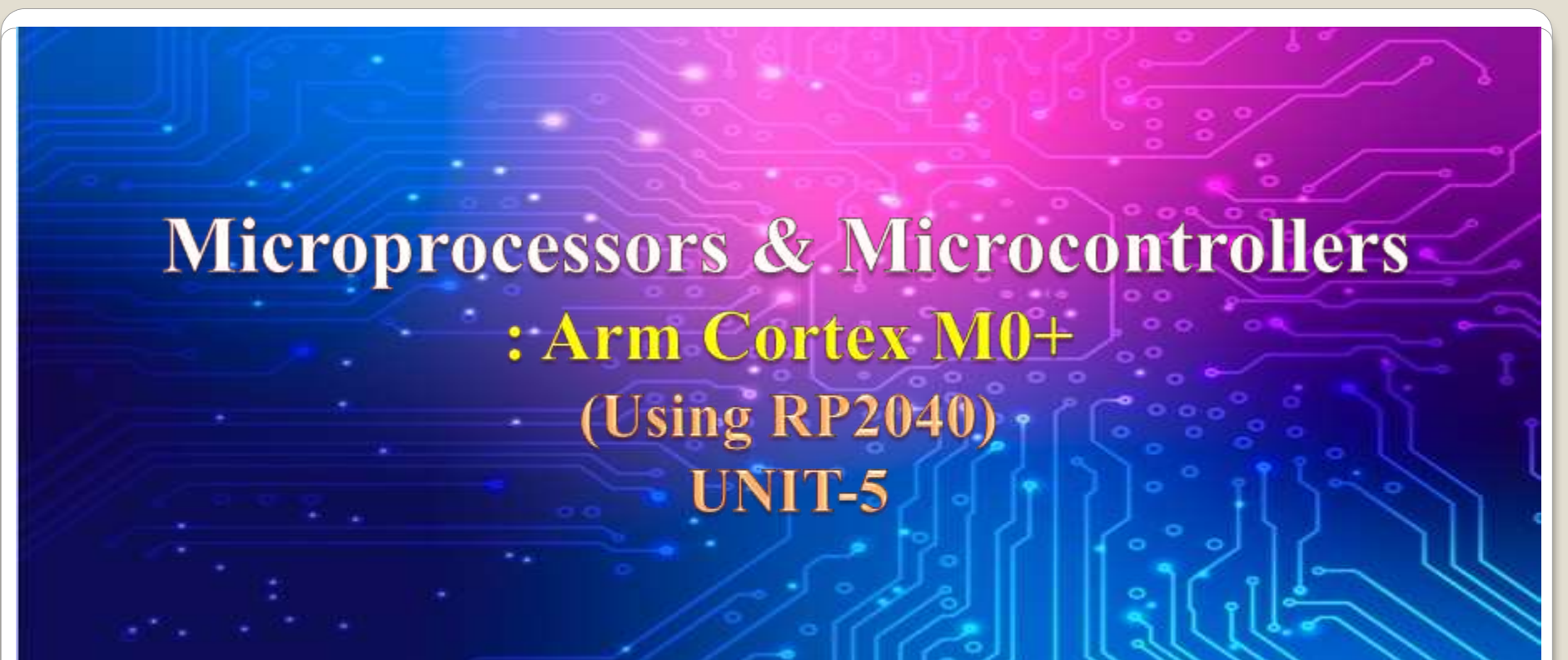


Embedded System and Microcontroller



Microprocessors & Microcontrollers

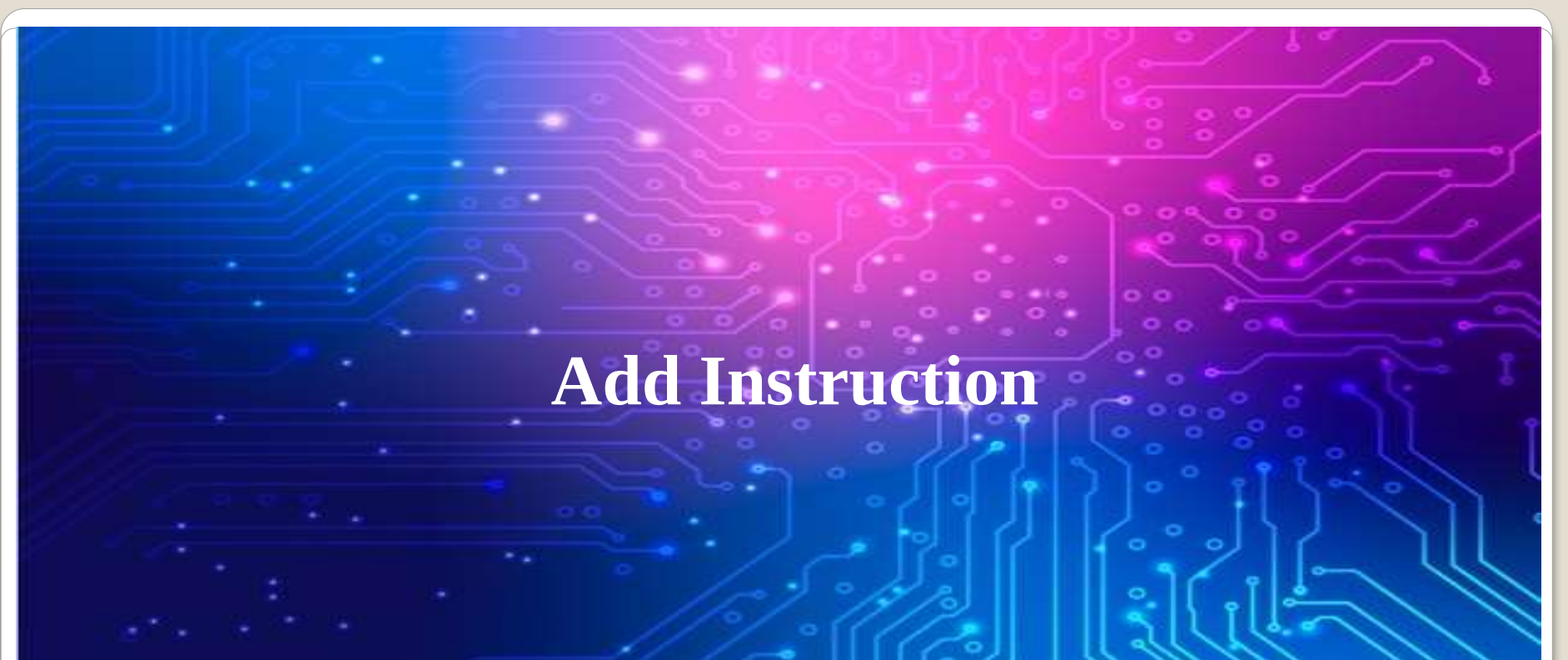
: Arm Cortex M0+

(Using RP2040)

UNIT-5

Add instruction and condition flags

Mouli Sankaran



Add Instruction

Technical Reference Manual of Cortex-M0+-(Ref 3)

Table 3-1 Cortex-M0+ instruction summary

Operation	Description	Assembler	Cycles
Move	8-bit immediate	MOV _S Rd, #<imm>	1
	Lo to Lo	MOV _S Rd, Rm	1
	Any to Any	MOV Rd, Rm	1
	Any to PC	MOV PC, Rm	2
Add	3-bit immediate	ADD _S Rd, Rn, #<imm>	1
	All registers Lo	ADD _S Rd, Rn, Rm	1
	Any to Any	ADD Rd, Rd, Rm	1
	Any to PC	ADD PC, PC, Rm	2
	8-bit immediate	ADD _S Rd, Rd, #<imm>	1
	With carry	ADCS Rd, Rd, Rm	1
	Immediate to SP	ADD SP, SP, #<imm>	1
	Form address from SP	ADD Rd, SP, #<imm>	1
	Form address from PC	ADR Rd, <label>	1

Note: Do not add the **S** suffix to the instructions
It is **not supported** on **RP2040** only without S suffix.
It is not supported in **Thumb 16** mode.

Lo regs: R0 to R7

Technical Reference Manual of Cortex-M0+-(Ref 3)

Page: 28/51

State change	Supervisor Call	SVC #<imm>	_ f
	Disable interrupts	CPSID i	1
	Enable interrupts	CPSIE i	1
	Read special register	MRS Rd, <specreg>	3
	Write special register	MSR <specreg>, Rn	3
	Breakpoint	BKPT #<imm>	_ f

Note: Special Registers

It also includes Program Status Register (**PSR**) register. We see the contents of PSR next.



Program Status Register (PSR)

prog_lec_2_2_3.ino and main_asm.S

ARM-Cortex-M0+ Devices

Generic User Guide

(Ref 4)

Program Status Register **Page: 66/113**

The *Program Status Register* (PSR) combines:

- *Application Program Status Register* (APSR)
- *Interrupt Program Status Register* (IPSR).
- *Execution Program Status Register* (EPSR).

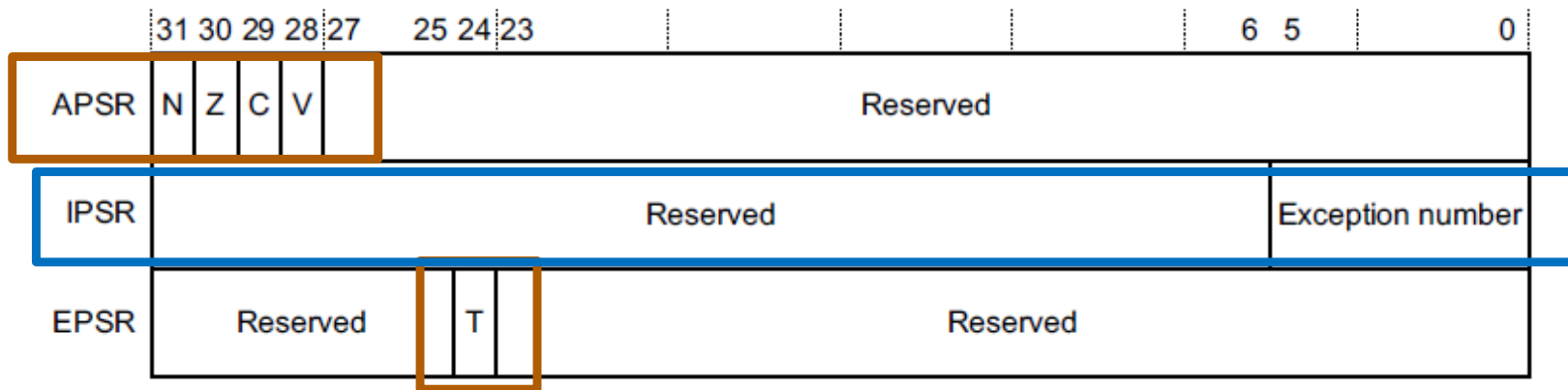
These registers are allocated as mutually exclusive bitfields within the 32-bit PSR. The PSR bit assignments are:

Table 2-4 APSR bit assignments

Bits	Name	Function
[31]	N	Negative flag.
[30]	Z	Zero flag.
[29]	C	Carry or borrow flag.
[28]	V	Overflow flag.
[27:0]	-	Reserved.

Table 2-6 EPSR bit assignments

Bits	Name	Function
[31:25]	-	Reserved.
[24]	T	Thumb state bit.
[23:0]	-	Reserved.



Note: Special Registers
include Program Status Register (PSR)
register. It stores the conditional flags NZCV flags

Note: We will look at the **IPSR** later



Demo of Asm Program

prog_lec_2_2_3.ino and main_asm.S

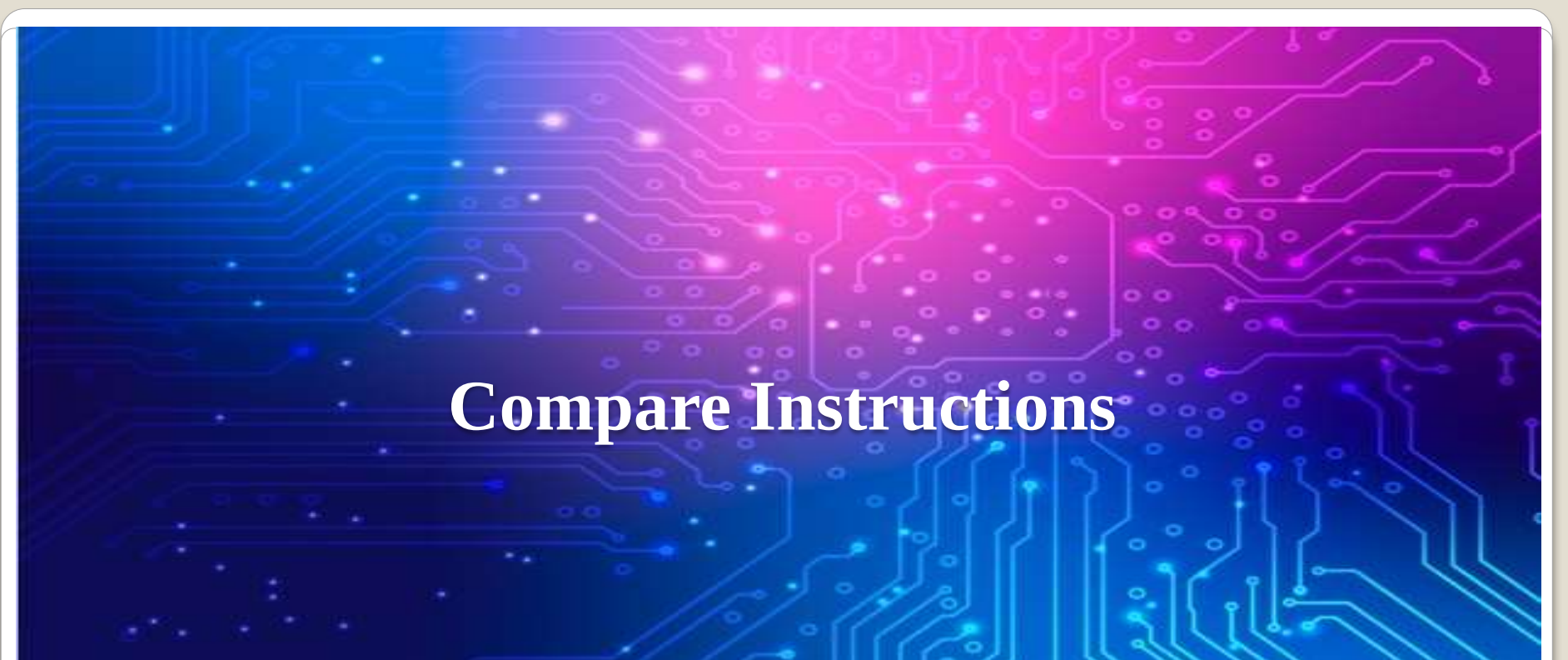


Microprocessors & Microcontrollers

: Arm Cortex M0+

(Using RP2040)

cmp and conditional branch instructions



Compare Instructions

Technical Reference Manual of Cortex-M0+-(Ref 3)



Page: 26/51

Compare	Compare	CMP Rn, Rm	1
	Negative	CMN Rn, Rm	1
	Immediate	CMP Rn, #<imm>	1

- These instructions compare the value in a register with either the value in another register or an immediate value.
- They update the condition flags on the result, but do not write the result to a register.
- The **CMP** instruction subtracts either the value in the register specified by **Rm**, or the immediate **imm** from the value in **Rn** and updates the flags. This is the same as a SUB instruction, except that the result is discarded.
- The **CMN** instruction adds the value of **Rm** to the value in **Rn** and updates the flags. This is the same as an **ADD** instruction, except that the result is discarded



Conditional branch Instructions

Technical Reference Manual of Cortex-M0+-(Ref 3)



Branch	Conditional	B<cc> <label>	1 or 2 ^e
e. 2 cycles if the branch is taken, one cycle if not-taken.	Unconditional	B <label>	2
	With link	BL <label>	3
	With exchange	BX Rm	2

- **B<cc> <label>**: **Conditional branch** is based on the condition mentioned in *cc*, branch to the target *label*
- **B <label>** : **Unconditional branch**, moves the control to the *label* always
- **BL <label>** : Branch to the label, after copying the return address into the Link register (return address)
- **BX Rm** : If **Rm** is **LR**, returns from a function call. The Rm could be any other register also which has the return address

cc above refers to the predefined condition codes defined by Arm

Let us see
them next ...

ARM Conditions Code (cc)

Condition Code	Interpretation	Status Flag State
EQ	Equal / equals zero	Z set
NE	Not equal	Z clear
CS / HS	Carry set / unsigned higher or same	C set
CC / LO	Carry clear / unsigned lower	C clear
MI	Minus / negative	N set
PL	Plus / positive or zero	N clear
VS	Overflow	V set
VC	No overflow	V clear
HI	Unsigned higher	C set and Z clear
LS	Unsigned lower or same	C clear or Z set
GE	Signed greater than or equal	N equals V
LT	Signed less than	N is not equal to V
GT	Signed greater than	Z clear and N equals V
LE	Signed less than or equal	Z set or N is not equal to V
AL	Always (optional)	Any

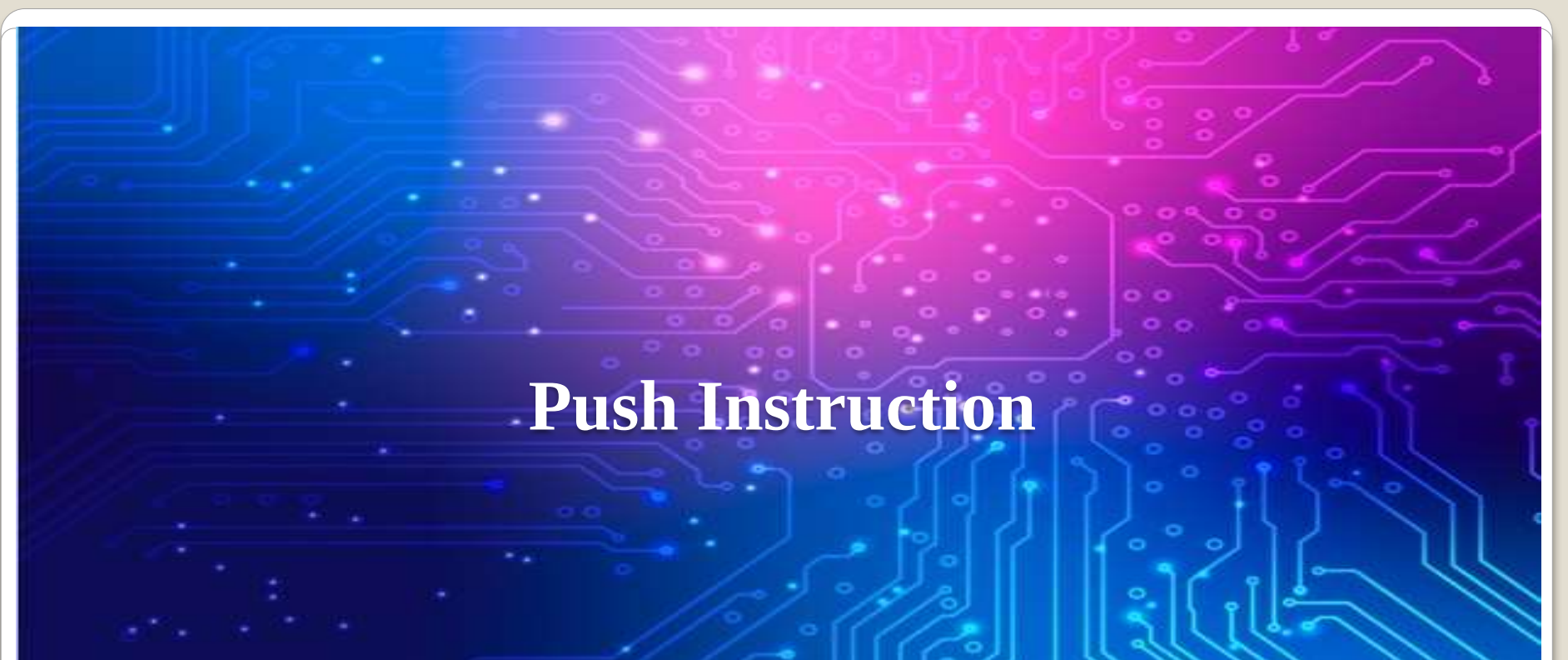
[Source link:](#)

Note: The **NZCV** flags are affected by the **CMP** instruction.



Demo of Asm Program

prog_lec_2_4_1.ino and main_asm.S



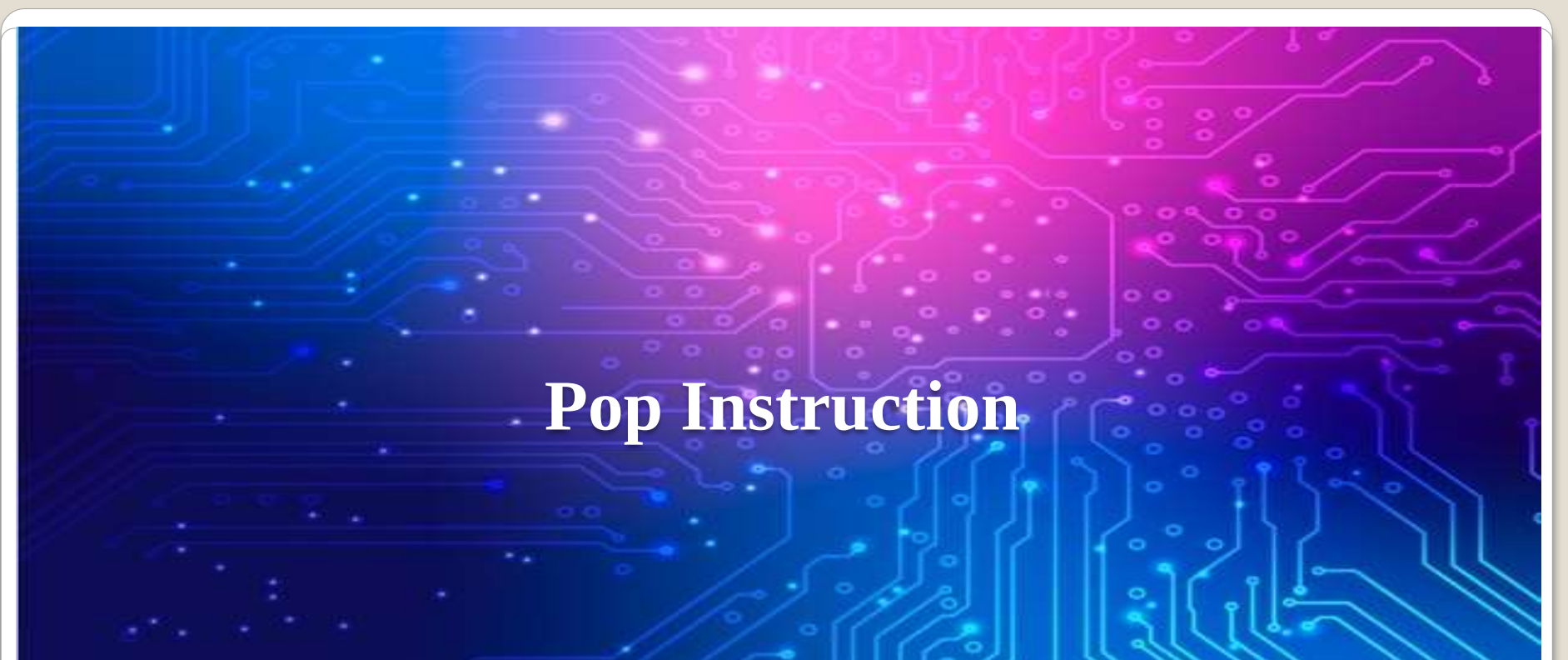
Push Instruction

Technical Reference Manual of Cortex-M0+-(Ref 3) – Push

Ref 3:
Page: 28/5

Operation	Description	Assembler	Cycles
Push	Push	PUSH {<loreglist>}	1+N ^c
	Push with link register	PUSH {<loreglist>, LR}	1+N ^d
Pop	Pop	POP {<loreglist>}	1+N ^c
	Pop and return	POP {<loreglist>, PC}	3+N ^d

- **Push** stores the registers onto a **full-descending stack**.
- **PUSH** stores registers on the stack, with the lowest numbered register using the lowest memory address and the highest numbered register using the highest memory address.
- **PUSH** uses the value in the SP register minus four as the **highest** memory address.
- On completion, PUSH updates the SP register to point to the location of the lowest stored value.



Pop Instruction

Operation	Description	Assembler	Cycles
Pop	Pop	POP {<loreglist>}	$1+N^c$
	Pop and return	POP {<loreglist>, PC}	$3+N^d$

- **POP** takes registers off a **full-descending stack**.
- **POP** loads registers from the stack, with the lowest numbered register using the lowest memory address and the highest numbered register using the highest memory address.
- **POP** uses the value in the SP register as the **lowest** memory address, implementing a full-descending stack.
- On completion, POP updates the SP register to point to the location above the highest location loaded from.
- If a POP instruction includes PC in its *reglist*, a branch to that location is performed when the POP instruction gets completed.



Demo of Asm Program

prog_lec_3_1_2.ino and main_asm.S



Microprocessors & Microcontrollers **: Arm Cortex M0+** **(Using RP2040)**

Declaring Variables in Assembly program



Declaring Variables in Assembly

Declaring Variables in Assembly

In the ASM program

```
@ Array of ARR_SIZE integer elements which are uninitialized  
myAsmVarArr1: .skip ARR_SIZE*4 @ 4 integers of size 4 bytes each  
myAsmVarArr2: .skip ARR_SIZE*4 @ 4 integers of size 4 bytes each
```

In the C program

```
extern int myAsmVarArr1[];  
extern int myAsmVarArr2[];
```

Note: The size is as per the declaration within the ASM module. The C program should be aware of the size of the array labels and access it accordingly.

- Define the variables in the assembly module
- Export the labels from the assembly module using **.global** directive
- This allows the linker to bind those modules if they need to be used by other modules
- Declare those variables as extern in the C program, which get resolved during the linking time



Demo of Asm Program

prog_lec_3_1_3.ino and main_asm.S



Microprocessors & Microcontrollers **: Arm Cortex M0+** **(Using RP2040)**

Interrupts, ISR and VTOR



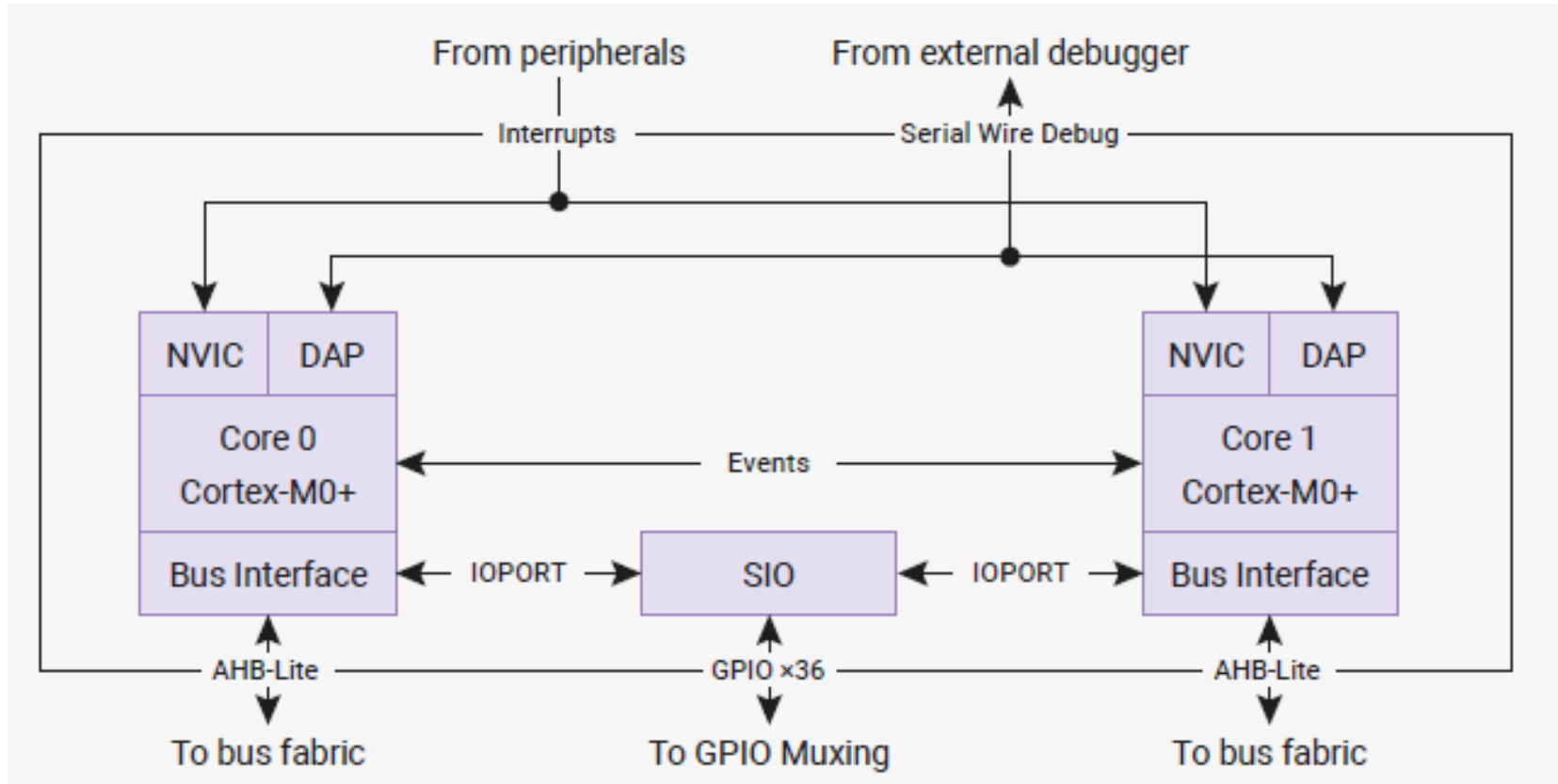
Interrupts and Exceptions

Interrupts and Exceptions



- **Interrupts** and **exceptions** are special kinds of control transfers
- They work somewhat like unprogrammed calls to subroutines
- They **alter** the **normal program flow** to handle **external events** or to **report errors** or **exceptional conditions**
- The **difference** between **interrupts** and **exceptions** is that **Interrupts** are used to handle asynchronous events external to the processor
 - **IRQs** from Peripherals, **Reset**, etc.
- But **exceptions** handle conditions detected by the processor itself in the course of executing instructions
 - Hard Fault due to Data alignment mismatch or Undefined Instruction, etc.

RP2040: System Overview



NVIC: Nested Vectored Interrupt Controller

DAP: Debug Access Port

NVIC: Cortex-M0+

NVIC: Nested Vectored Interrupt Controller

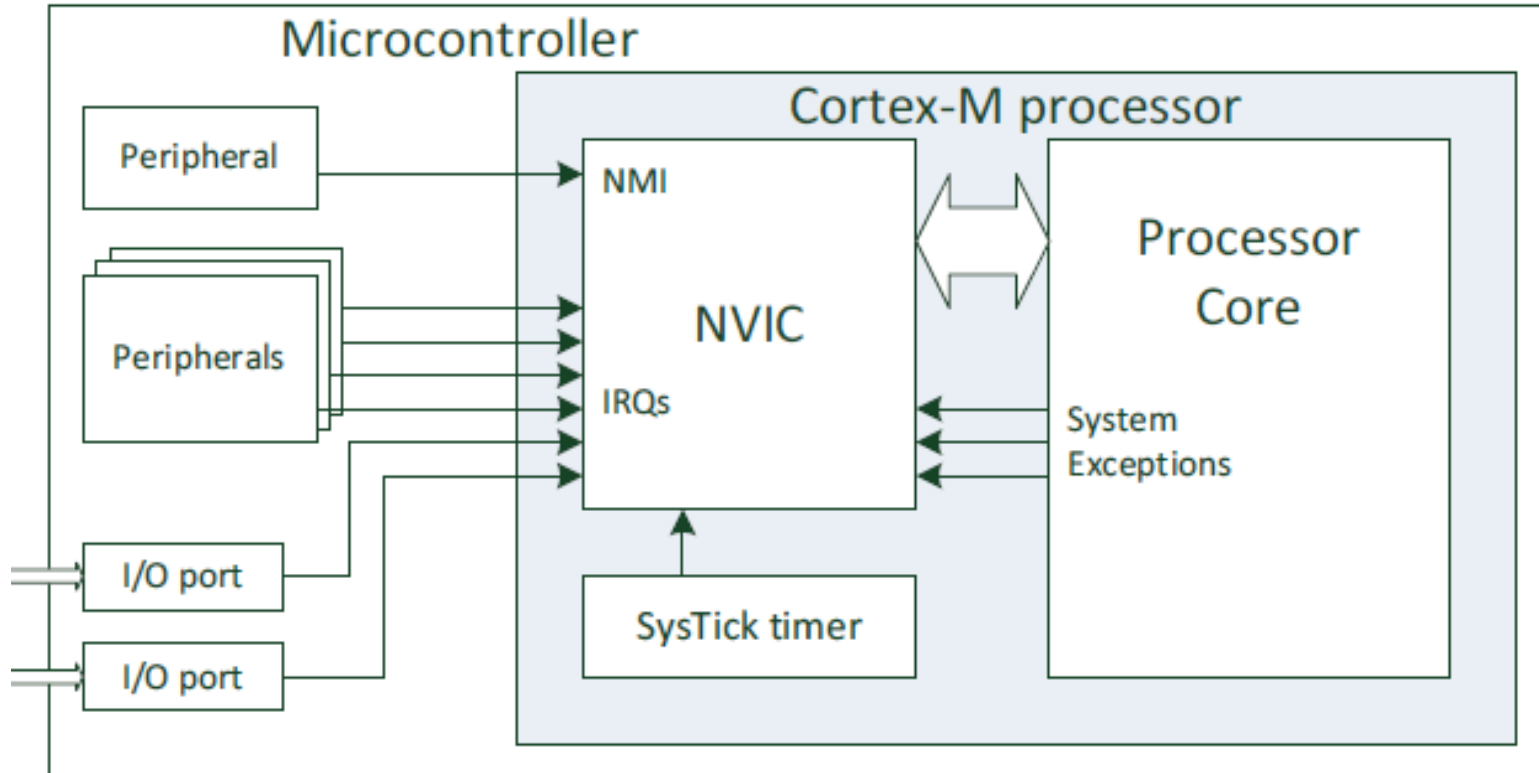


Figure 8.2

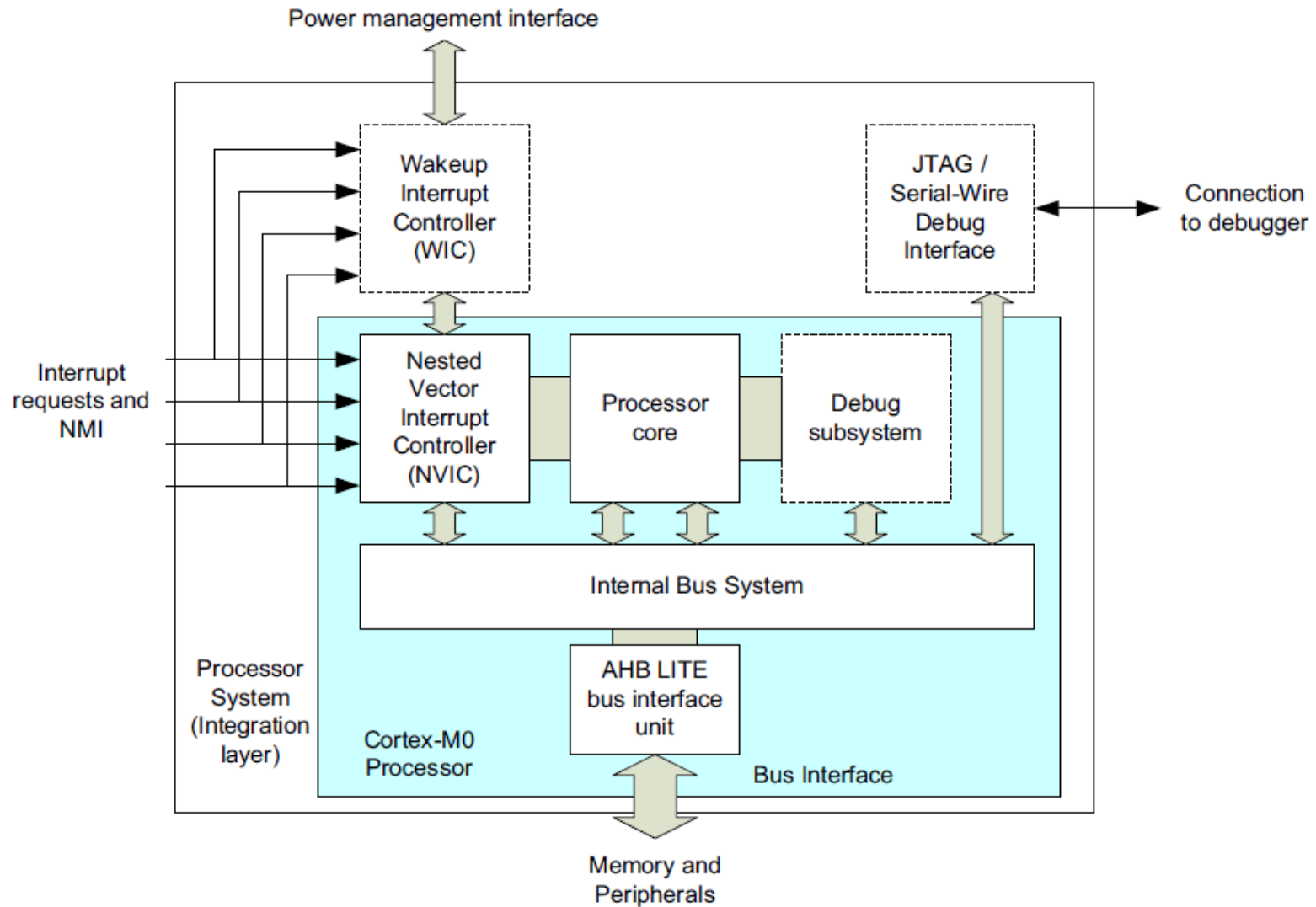
The NVIC in the Cortex[®]-M0 and Cortex-M0+ processors can deal with up to 32 IRQ inputs, an NMI, and a number of system exceptions.

NVIC in Cortex-M0+

- The Cortex-M processors are designed to provide very quick and deterministic interrupt responses.
- Processor's execution control part is closely coupled with a built-in Nested Vectored Interrupt Controller (NVIC) to get a faster responses.
- The NVIC provides powerful and yet easy-to-use interrupt management.
- It supports up to 32 interrupt requests from various peripherals (chip design dependent).
- Nested" refers to the fact that interrupts can themselves be interrupted, by higher-priority interrupts.
- "Vectored" refers to the hardware dispatching each interrupt to a distinct handler routine, specified by the vector table.
- NVIC registers (for enabling and configuring the priorities of interrupts) are only accessible using word transfers.

NVIC and WIC

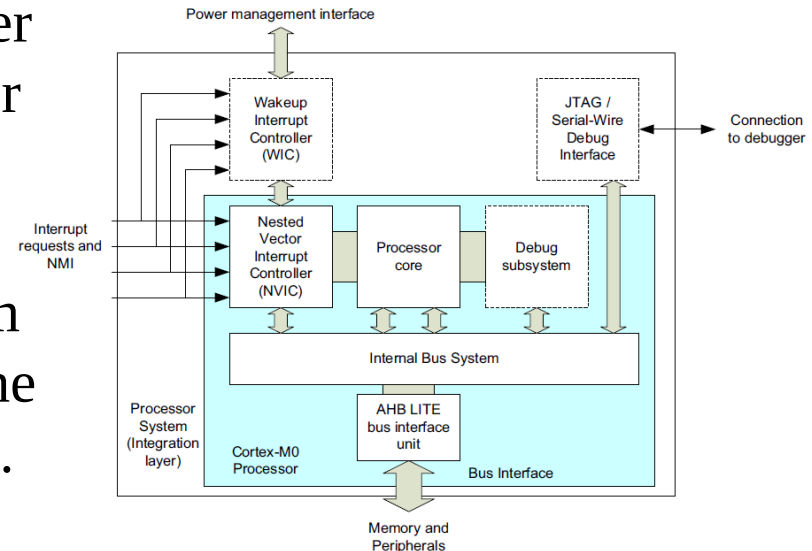
WIC: Wake up Interrupt Controller



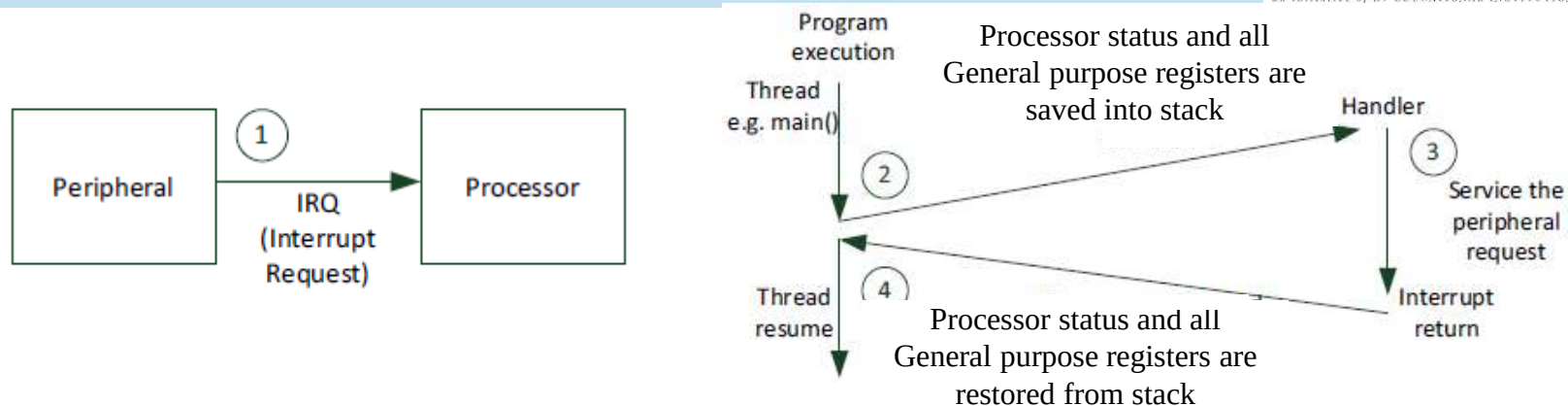
NVIC and WIC

WIC: Wake up Interrupt Controller

- The WIC is an optional unit. In low-power applications, the microcontroller can enter standby state with most parts of the processor powered down.
- Under this situation, the WIC can perform the function of interrupt masking while the NVIC and the processor core are inactive.
- RP2040 implementation includes a WIC.
- When an interrupt request is detected, the WIC informs the power management to power up the system so that the NVIC and the processor core can then handle the rest of the interrupt processing.
- This enables the processor and NVIC to be put into a very low-power sleep mode leaving the WIC to identify and prioritize interrupts.
- The processor fully implements the Wait For Interrupt (WFI), Wait For Event (WFE) and the Send Event (SEV) instructions.

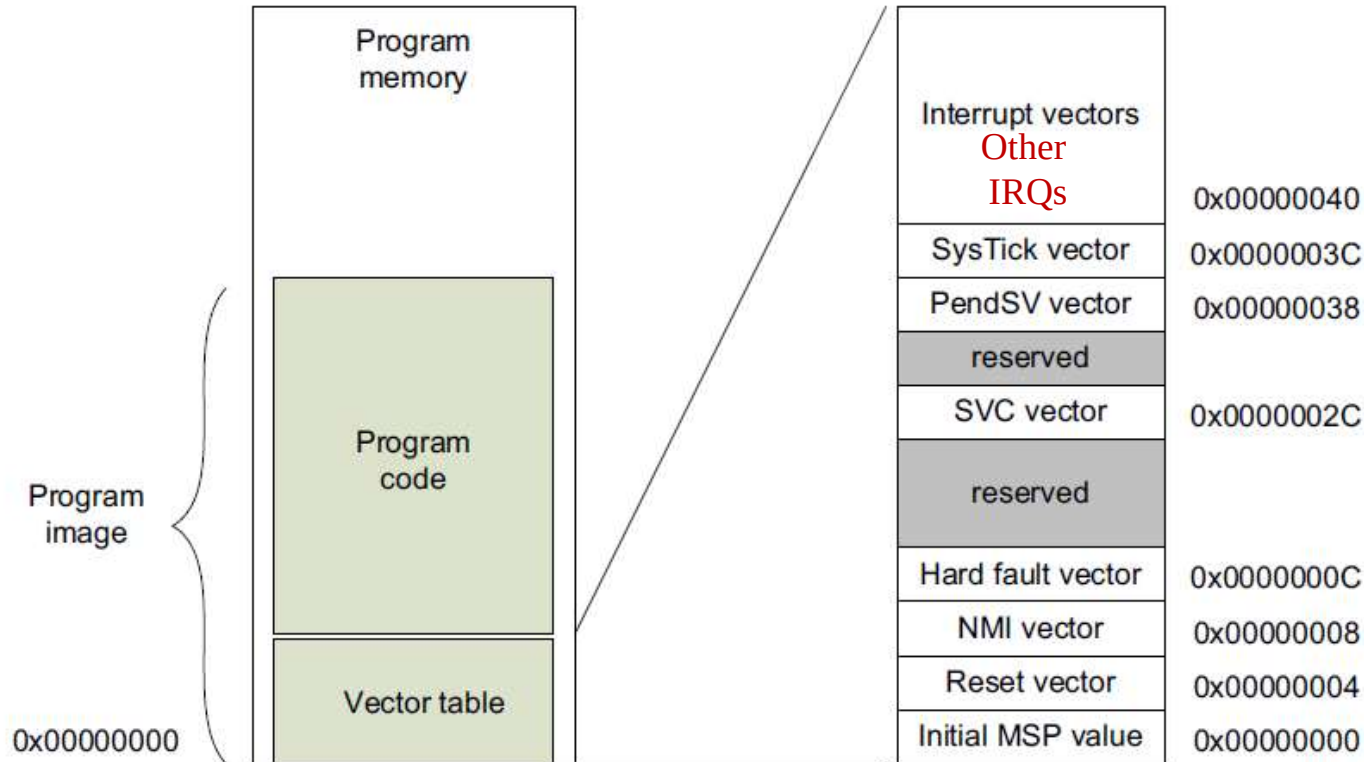


Interrupts and Exceptions



1. A peripheral generates an interrupt request (IRQ) to the processor.
2. The NVIC receives and decides based on the configuration whether to interrupt the CPU or not.
3. The CPU detects and accepts the interrupt coming in from the NVIC.
4. CPU completes the execution of the current instruction. If multi-cycle instruction (LDM/STM) the execution is terminated and continued from the beginning after the interrupt is serviced.
5. CPU saves the Status Reg (PSR), all registers along with PC into the stack.
6. CPU locates the starting address of the ISR or handler corresponding to the IRQ using the Interrupt Vector Table and executes it.
7. The processor finishes the handler execution, restores the information previously pushed to the stack, and resumes the interrupted program.

Interrupt Vector Table in ROM on Reset



Each entry, points to the ISRs of that particular interrupt Or exception

On Reset

VTOR

Figure 4.15
Vector table in a program image.

- The Interrupt vector table is initially located at reset location 0x0 in the ROM, later this will be moved to Flash or SRAM based on the boot ROM code.

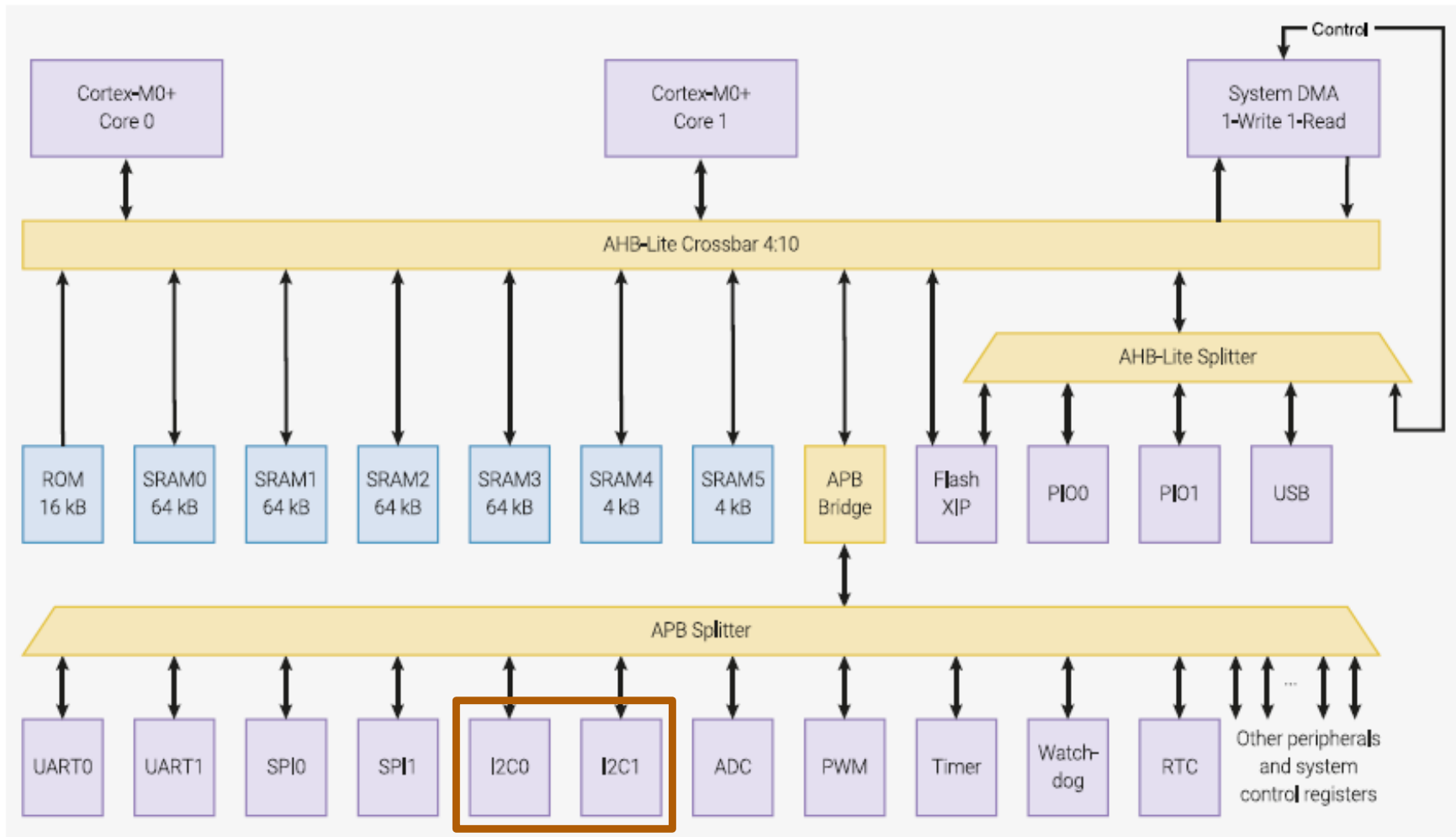


I2C in RP2040

[illegible]

There are many pins that can be mapped to I2C devices to enable flexible interfacing of other ICs in the board.

I2C Interface with the APB Bus (low speed)



Ref 5: RP2040 Data sheet

Page: 460/647, Section 4.3

RP2040: Address Mapping of Peripherals and I2C

ROM	0x00000000
XIP	0x10000000
SRAM	0x20000000
APB Peripherals	0x40000000
AHB-Lite Peripherals	0x50000000
IOPORT Registers	0xd0000000
Cortex-M0+ internal registers	0xe0000000

APB Peripherals:

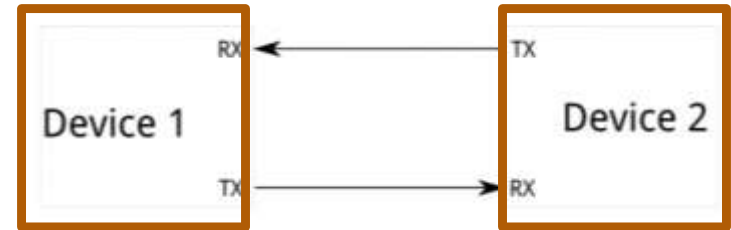
I2C0_BASE	0x40044000
I2C1_BASE	0x40048000

I2C Introduction



Problems with Serial Port

- Because serial ports are asynchronous (no clock is transmitted), devices using them must agree ahead of time on a data rate.

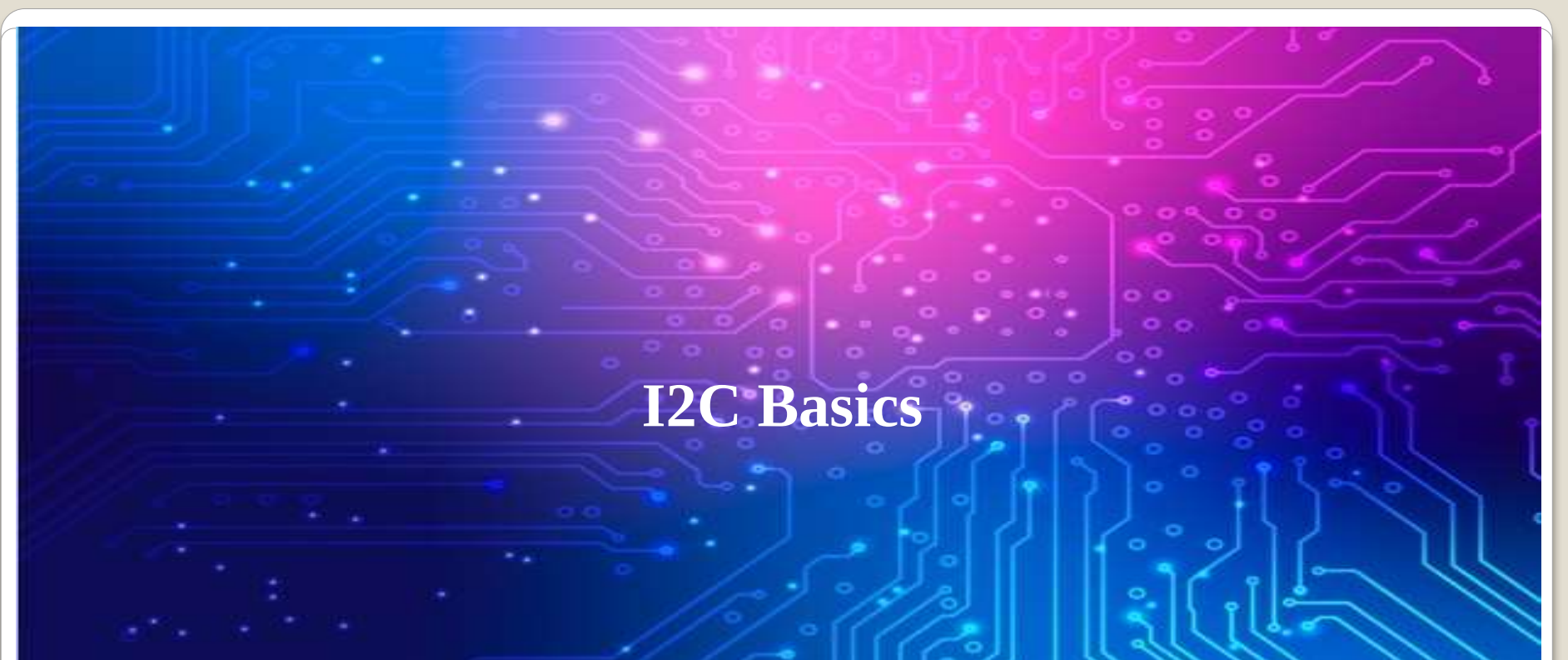


- The two devices must also use the same data rate (bauds), if they are different, garbled data reception results.
- Need for an UART chip on both ends of the channel, is an overhead.
- For every 8 bits of data minimum two additional (start and stop) bits are required for every 8 bits data, eating up the data rate of the channel.
- Other major drawback is that only two devices can talk to each other, when there are requirements to connect multiple devices, UART is not suitable.
- Finally, data rate is an issue. While there is no theoretical limit to asynchronous serial communications, most UART devices are built to only support a certain set of fixed baud rates, to take care of inter-operability between devices from different vendors.
 - And the highest of these is usually around **2,30,400 bits per second (bauds)**.

UART: Universal Asynchronous Transmitter Receiver

I2C : Introduction

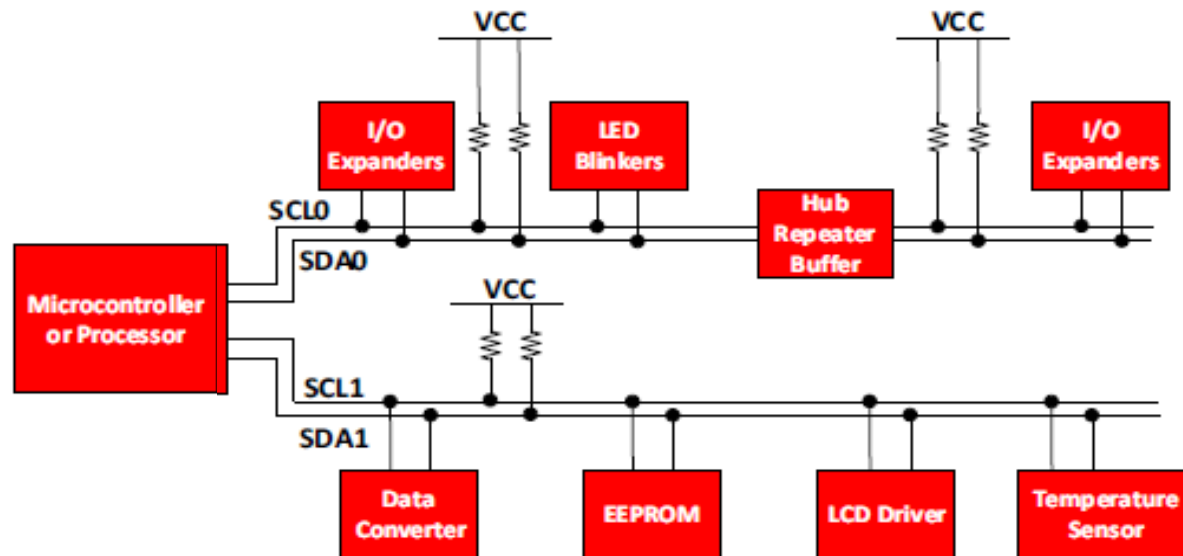
- I²C (Inter-Integrated Circuit, eye-squared-C), alternatively known as I²C or IIC, is a:
 - synchronous,
 - multi-controller/multi-target,
 - packet switched,
 - single-ended,
 - serial communication bus
- It was invented in 1982 by Philips Semiconductors.
- It is widely used for attaching lower-speed peripheral ICs to processors and microcontrollers in short-distance, intra-board communication



I2C Basics

I2C Bus: How is it used?

(One typical usage scenario)

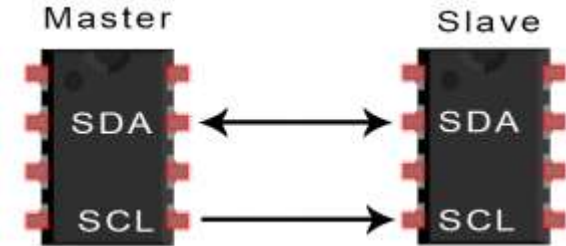


Note: This is to connect multiple sensors (slave devices) to a control processor (master device)

Mainly used for communication between Microcontrollers and other devices like, EEPROM, Real-Timer, interface chips, LCD driver, A/D converter, etc.

I2C : Basics

- I2C combines the best features of SPI (Serial Peripheral Interface) and UARTs.
- With I²C, we can connect multiple slave devices to a single master (like SPI) and we can have multiple masters controlling/exchanging information with single, or multiple slave devices on a bus.
- This is really useful when we want to have more than one microcontroller logging data to a single memory card or displaying text to a single LCD.
- I2C uses only two wires to exchange data between devices. No need for GND line.
- **SDA** is for the devices on the bus to exchange data between them.
- **SCL** is line that carries the clock signal to all the connected devices to have a synchronous exchange of data.

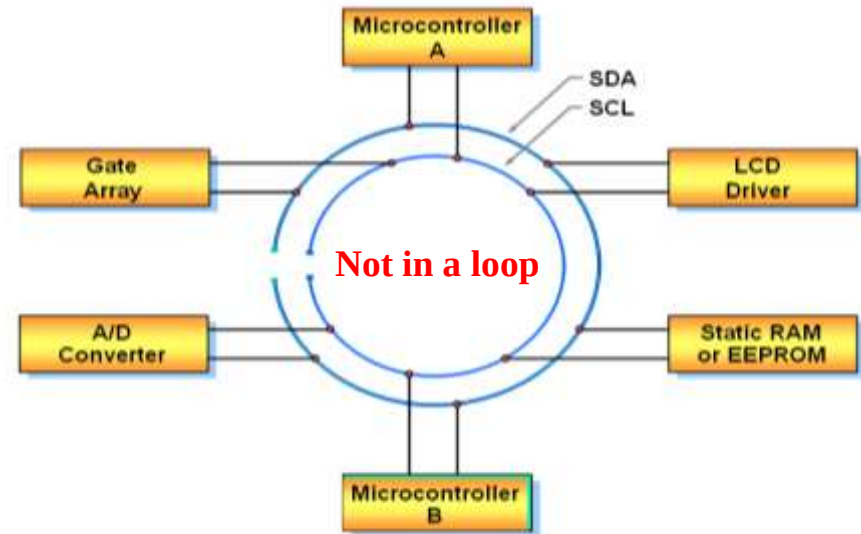


SDA: Serial Data
SCL: Serial Clock

Note: The I²C literature uses Master/Slave nomenclature to represent the device which is in control of the transmission on the shared bus, at any point in time. The SCL (clock) is generated by the device which is configured to be a master.

I2C Bus Definitions and A Sample Configuration

- **Master:**
 - Initiates a transfer by generating start and stop conditions
 - Generates the clock
 - Transmits the slave address
 - Determines data transfer direction
- **Slave:**
 - Responds only when addressed
 - Timing is controlled by the clock line



I2C : Specification

Wires Used	2
Maximum Speed	Standard mode= 100 kbps Fast mode= 400 kbps High speed mode= 3.4 Mbps Ultra fast mode= 5 Mbps
Synchronous or Asynchronous?	Synchronous
Serial or Parallel?	Serial
Max # of Masters	Unlimited
Max # of Slaves	1008

Note: Slave devices address uses a 7-Bit address but the bus specification allows an extension to 10 bits as well.



Microprocessors & Microcontrollers **: Arm Cortex M0+** **(Using RP2040)**

Introduction to DMA



DMA in RP2040

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Raspberry Pi Pico Pin Reference

Legend:

- Power/Ground
- UART (Serial)
- Analog
- GPIO/PIO
- PWM
- SPI
- I2C
- Control/Debug

Pin Functions:

Pin	Function	Pin	Function
1	TX0	40	VBUS
2	RX0	39	VSYS
3	GND	38	GND
4	SDA1	37	3V3_EN
5	SCL1	36	3V3
6	TX1	35	ADC_VREF
7	RX1	34	GP28
8	GND	33	GND
9	SDA1	32	GP27
10	SCL1	31	GP26
11	TX1	30	RUN/RESET
12	RX1	29	GP22
13	GND	28	GND
14	SDA1	27	GP21
15	SCL1	26	GP20
16	TX0	25	GP19
17	RX0	24	GP18
18	GND	23	GND
19	SDA1	22	GP17
20	DO NOT USE	21	GP16

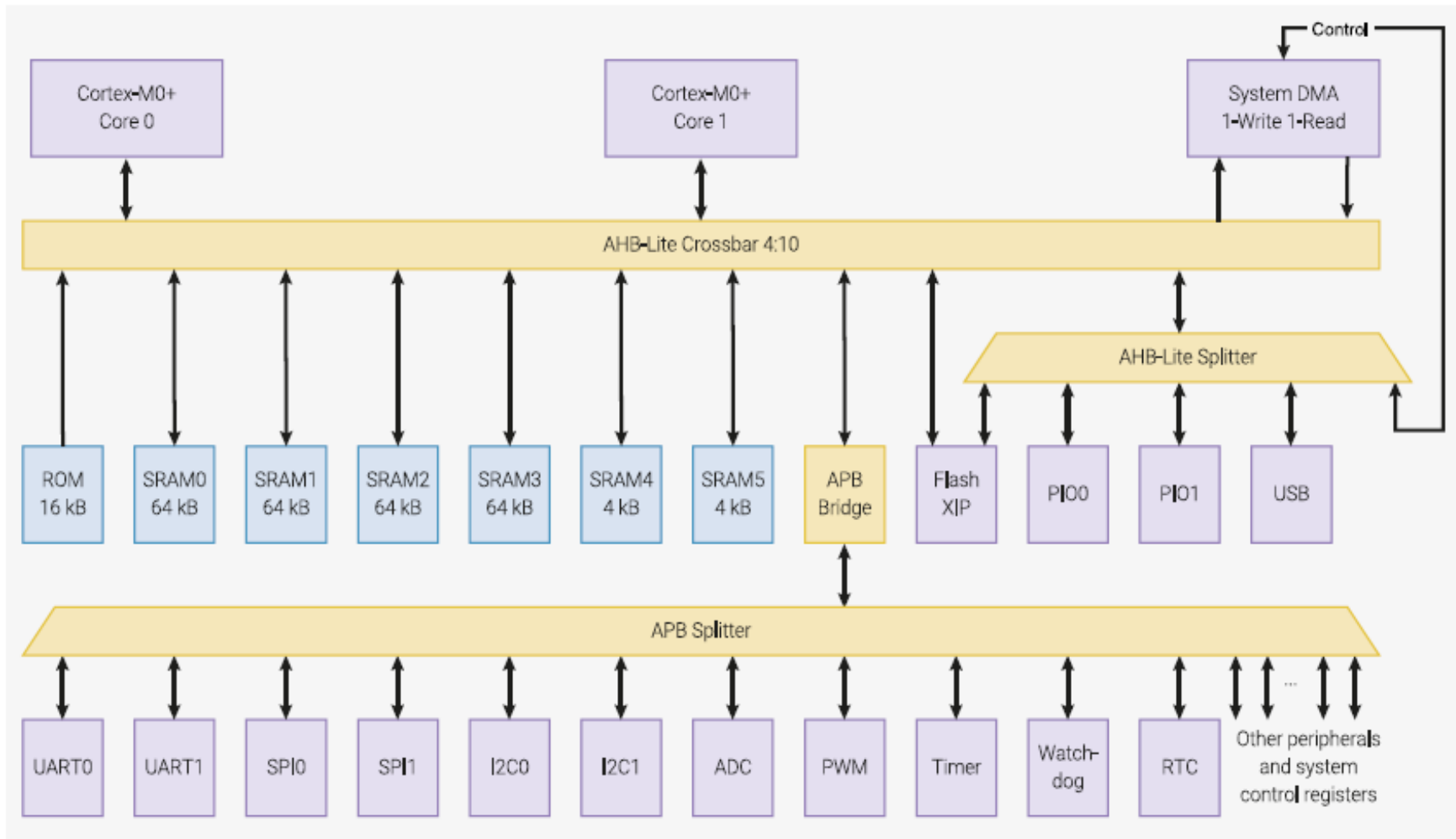
Additional Pin Functions:

Pin	Function	Pin	Function
GP0	PWM0A	DI1	GP28_A2
GP1	PWM0B	DO1	GP27_A1
GP2	PWM1A	SCK1	GP26_A0
GP3	PWM1B	SCK0	SDA1
GP4	PWM2A	CS0	SCLO
GP5	PWM2B	CS1	SCLO
GP6	PWM3A	CS0	SCLO
GP7	PWM3B	CS1	SCLO
GP8	PWM4A	CS0	SCLO
GP9	PWM4B	CS1	SCLO
GP10	PWM5A	CS0	SCLO
GP11	PWM5B	CS1	SCLO
GP12	PWM6A	CS0	SCLO
GP13	PWM7B	CS1	SCLO
GP14	PWM7A	CS0	SCLO
GP15	DO NOT USE	CS1	SCLO

Adafruit

Reason: RP2040
Meant for low power
Applications and
bringing DMA signals
out would increase
the pins and its power
budget and size too.

DMA Interface with the Bus Fabric



Ref 5: RP2040 Data sheet

Page: 16/647, Figure 4

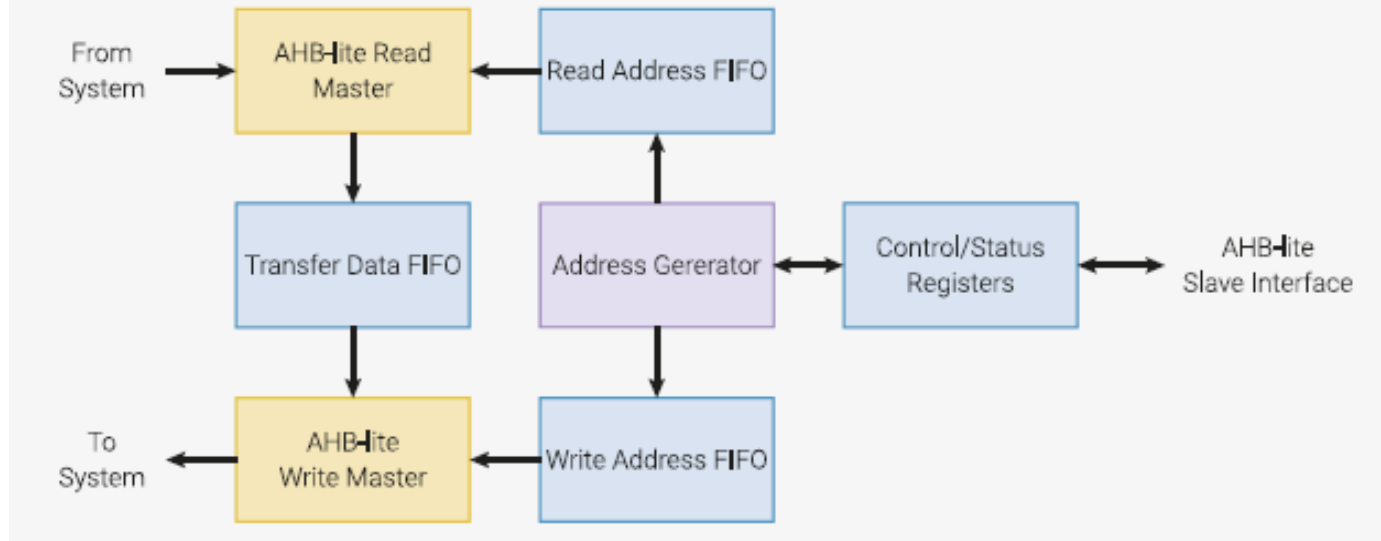
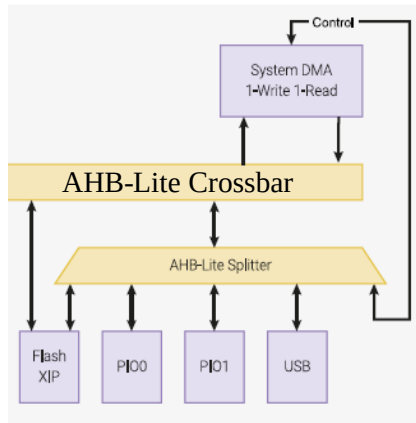
RP2040: Address Mapping of Peripherals and DMA

ROM	0x00000000
XIP	0x10000000
SRAM	0x20000000
APB Peripherals	0x40000000
AHB-Lite Peripherals	0x50000000
IOPORT Registers	0xd0000000
Cortex-M0+ internal registers	0xe0000000

AHB-Lite peripherals:

DMA_BASE	0x50000000
----------	------------

DMA: Block



- DMA controller has separate read and write master connections to the bus fabric, and performs bulk data transfers on a processor's behalf.
- This leaves processors free to attend to other tasks, or enter low-power sleep states.
- The data throughput of the DMA is also significantly higher than RP2040 cores.

DMA: Explained

- The DMA can perform one read access and one write access, up to 32 bits in size, every clock cycle.
- There are **12 independent channels**, each which supervise a sequence of bus transfers, usually in one of the following scenarios:
 - **Memory-to-peripheral**: a peripheral signals the DMA when it needs more data to transmit. The DMA reads data from an array in RAM or flash, and writes to the peripheral's data FIFO.
 - **Peripheral-to-memory**: a peripheral signals the DMA when it has received data. The DMA reads this data from the peripheral's data FIFO, and writes it to an array in RAM.
 - **Memory-to-memory**: the DMA transfers data between two buffers in RAM, as fast as possible.
- Each channel has its own control and status registers (CSRs), with which software can program and monitor the channel's progress.
- When multiple channels are active at the same time, the DMA shares bandwidth evenly between the channels, with round-robin over all channels which are currently requesting data transfers.

DMA: More Details

- The transfer size can be either 32, 16, or 8 bits.
- This is configured once per channel: source transfer size and destination transfer size are the same.
- The DMA performs standard byte lane replication on narrow writes, so byte data is available in all 4 bytes of the data bus, and halfword data in both halfwords.
- Channels can be combined in varied ways for more sophisticated behaviour and greater autonomy.
- Making the DMA more autonomous means that much less processor supervision is required: overall this allows the system to do more at once, or to dissipate less power

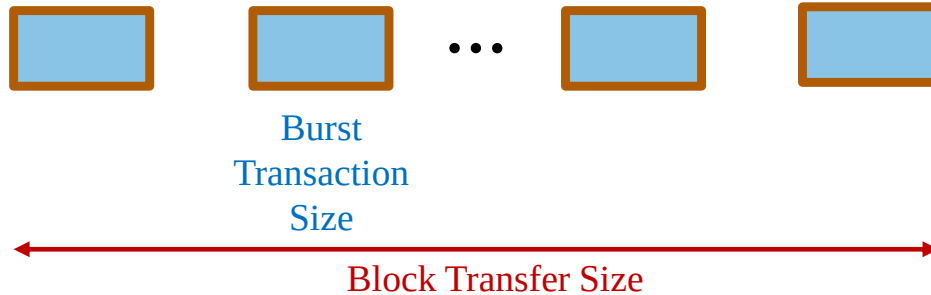
DMA: Channel Configuration

- Each channel has four control/status registers:
 - **READ_ADDR** is a pointer to the next address to be read from
 - **WRITE_ADDR** is a pointer to the next address to be written to
 - **TRANS_COUNT** shows the number of transfers remaining in the current transfer sequence, and is used to program the number of transfers in the next transfer sequence).
 - **CTRL** is used to configure all other aspects of the channel's behaviour, to enable/disable it, and to check for completion.
- These are **live registers**: they update continuously as the channel progresses
- Read and write address registers update automatically after each read/write access.
- They increment by 1, 2 or 4 bytes at a time, depending on the transfer size configured in CTRL.
- Software should generally program these registers with new start addresses each time a new transfer sequence starts.

Ref 5: RP2040 Data sheet

Page: 93/647, Section 2.5.1

DMA: Block Transfer and Burst Transaction Sizes



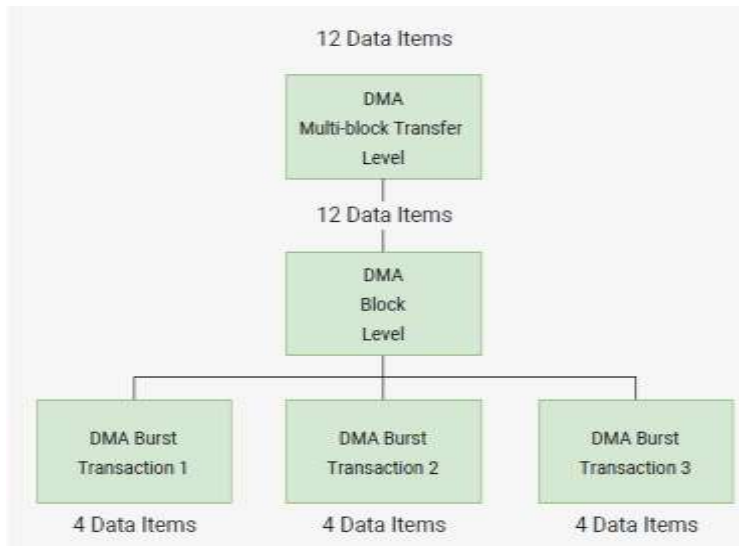
Between a **source** and a **destination**

Source and destination can be any part of memory and peripherals, which are provided with DMA transfer capability
Example: UART, USB, etc.

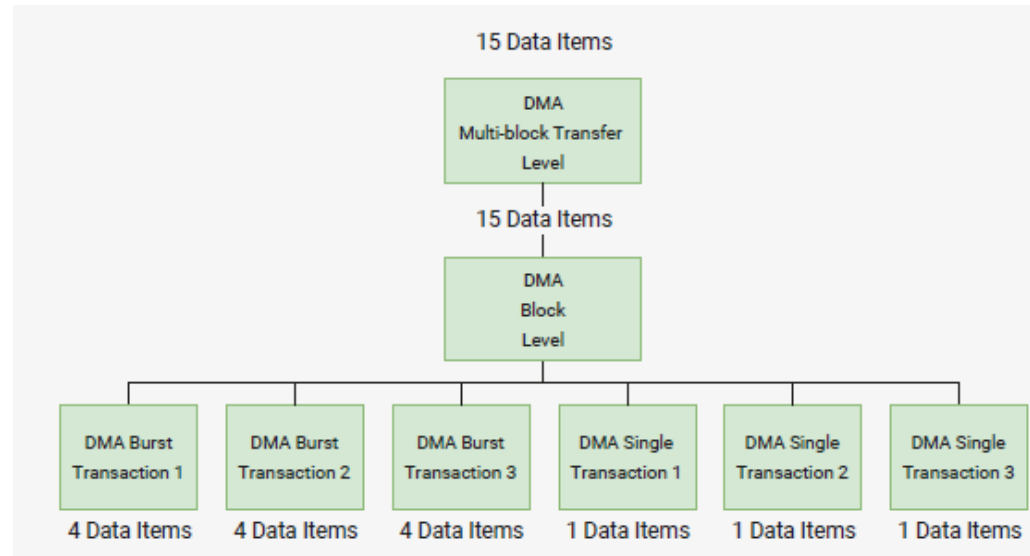
- The DMA Controller is programmed by the processor with the number of data items (block size) that are to be transmitted or received on every **block transfer**.
- The block is broken into a number of transactions.
- The DMA Controller must also be programmed with the number of data items to be transferred for **each DMA request**.
- This is known as the **burst transaction length**.
- Both the sizes are programmed while configuring the DMA for a transfer between chosen pair of devices (memory and/or peripherals) within RP2040.

Examples: DMA Transactions

- Total Block Transfer size : **12 Bytes**
- Burst Transaction size : **4 Bytes**



- Total Block Transfer size : **15 Bytes**
- Burst Transaction size : **4 Bytes**



Note: Burst transaction size is set to 4 bytes because AHB-lite fabric has 32-bits wide interconnections between the masters and slaves.

DMA IRQ

IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source
0	TIMER_IRQ_0	6	XIP_IRQ	12	DMA_IRQ_1	18	SPI0_IRQ	24	I2C1_IRQ
1	TIMER_IRQ_1	7	PIO0_IRQ_0	13	IO_IRQ_BANK0	19	SPI1_IRQ	25	RTC_IRQ
2	TIMER_IRQ_2	8	PIO0_IRQ_1	14	IO_IRQ_QSPI	20	UART0_IRQ		
3	TIMER_IRQ_3	9	PIO1_IRQ_0	15	SIO_IRQ_PROC0	21	UART1_IRQ		
4	PWM_IRQ_WRAP	10	PIO1_IRQ_1	16	SIO_IRQ_PROC1	22	ADC_IRQ_FIFO		
5	USBCTRL_IRQ	11	DMA_IRQ_0	17	CLOCKS_IRQ	23	I2C0_IRQ		

- Each core is equipped with a standard ARM Nested Vectored Interrupt Controller (NVIC) which has 32 interrupt inputs.
- Each NVIC has the same interrupts routed to it, with the exception of the GPIO interrupts.
- For the DMA, Any combination of channel interrupt requests can be routed to either system **DMA_IRQ 1** or 2.